Readme do Trabalho_1_PAA

Funcionalidades do Programa

Este programa compacta e descompacta arquivos de texto usando o algoritmo de Huffman. Suas principais funcionalidades são:

- **Interface de Menu:** O programa apresenta um menu interativo e simples, permitindo ao usuário escolher a operação desejada.
- **Dois Modos de Codificação:**
    1. **Codificação por Caracter:** Trata cada caractere individual do texto (incluindo espaços, pontuação e quebras de linha) como um símbolo. Este é o método clássico de Huffman.
    2. **Codificação por Palavra:** Trata cada palavra e cada sequência de delimitadores (espaços, pontuação) como um símbolo único. Este método pode oferecer taxas de compressão superiores para textos em linguagem natural.
- **Suporte a Unicode:** O programa foi construído para lidar corretamente com textos na língua portuguesa, incluindo todos os caracteres acentuados (ã, ç, é, etc.), graças ao uso de streams e tipos de dados de caracteres largos (wchar_t, wstring).
- **Compactação e Descompactação:** Para cada modo, compacta um arquivo de texto para um arquivo binário e descompactar este arquivo de volta ao seu estado original, garantindo que a descompressão seja sem perdas.
- **Cabeçalho Auto-contido:** O arquivo compactado gerado inclui um cabeçalho binário contendo a tabela de frequências (o dicionário). Isso torna o arquivo autossuficiente, permitindo que a função de descompactação reconstrua a árvore de Huffman exata sem precisar de nenhum arquivo externo.
- **Medição de Desempenho:** Ao final de cada operação, o programa exibe o tempo total de execução em segundos, permitindo uma análise de desempenho dos diferentes métodos de compactação.

Relatório de Desempenho

Este relatório analisa os resultados obtidos a partir de três arquivos de teste. Foram pegos vários livros da internet para formar os arquivos. O texto 3 contém o texto 2 que contém o texto 1.

1. Taxa de Compressão

Arquivo de Teste	Método	Tamanho Original (bytes)	Tamanho Compactado (bytes)	Taxa de Compressão
Texto 1	**Por caracter**	**8,957,952**	**5,140,480**	**42.62%**
	Por palavra	8,957,952	5,292,032	40.92%
Texto 2	**Por caracter**	**15,904,768**	**9,261,056**	**41.77%**
	Por palavra	15,904,768	9,035,776	43.19%
Texto 3	**Por caracter**	**33,071,104**	**19,230,720**	**41.85%**
	Por palavra	33,071,104	16,384,000	50.46%

2. Análise Empírica do Desempenho

A análise de desempenho considera a relação entre a **taxa de compressão (espaço)** e o **tempo de execução (velocidade)** para cada método.

Arquivo de Teste	Método	Tempo de Compactação	Tempo de Descompactação
Texto1	Por caracter	**2.3776 s**	**0.3928 s**
	Por palavra	3.2732 s	0.5879 s
Texto 2	Por caracter	**4.1579 s**	**0.6997 s**
	Por palavra	6.0781 s	1.0585 s
Texto 3	Por caracter	**8.6762 s**	**1.4344 s**
	Por palavra	12.3686 s	2.1093 s

Análise do Texto 1 (8.54 MB)

- **Taxa de Compressão:** Surpreendentemente, a codificação por **caracter** foi superior, com 42.62% de compressão contra 40.92% da codificação por palavra. Isso sugere que, para este arquivo específico, a redundância entre caracteres individuais era mais explorável do que a repetição de palavras inteiras. O custo de armazenar o dicionário de palavras (que é muito maior que o de caracteres) provavelmente superou os benefícios da compressão.
- **Tempo de Execução:** A codificação por caracter foi significativamente mais rápida, tanto na compactação quanto na descompactação.

Análise do Texto 2 (15.1 MB)

- **Taxa de Compressão:** Aqui, a codificação por **palavra** começa a mostrar sua vantagem, alcançando uma taxa de 43.19%, ligeiramente melhor que os 41.77% da

codificação por caracter. Com um texto maior, a probabilidade de repetição de palavras comuns aumenta.

- **Tempo de Execução:** A codificação por caracter continua sendo a mais rápida, com uma diferença considerável de quase 2 segundos na compactação.

Análise do Texto 3 (31.5 MB)

- **Taxa de Compressão:** Neste arquivo grande, a superioridade da codificação por **palavra** é clara e expressiva, atingindo uma excelente taxa de **50.46%**, quase 9 pontos percentuais acima da codificação por caracter. Isso demonstra que, em volumes maiores de linguagem natural, a repetição de palavras é o fator de redundância dominante.
- **Tempo de Execução:** A diferença de velocidade também aumenta. A codificação por palavra leva quase 4 segundos a mais para compactar. Isso ocorre devido ao custo computacional de gerenciar um dicionário com milhares de palavras únicas em vez de um dicionário com cerca de 100-200 caracteres únicos.

Conclusão da Análise

Existe um claro **trade-off entre velocidade e taxa de compressão**.

- A **Codificação por Caracter** é consistentemente mais rápida, pois lida com um "alfabeto" de símbolos muito menor, o que torna a construção da árvore de Huffman e a geração de códigos mais eficientes. Ela oferece uma boa compressão de base para qualquer tipo de texto.
- A **Codificação por Palavra** é mais lenta devido ao dicionário massivo de palavras que precisa gerenciar. No entanto, para arquivos de texto grandes e em linguagem natural, ela é capaz de explorar a redundância semântica (repetição de palavras) para alcançar taxas de compressão significativamente superiores, como visto no Texto 3.

A escolha do método ideal depende do objetivo: para máxima velocidade, a codificação por caracter é a melhor. Para máxima economia de espaço em arquivos de texto grandes, a codificação por palavra é a mais indicada, ao custo de um maior tempo de processamento.

3. Metodologia de Medição de Tempo

A mensuração do tempo de execução foi realizada utilizando as ferramentas da biblioteca padrão do C++ <chrono>.